

LAPORAN PROJECT CAPSTONE DATA SCIENCE

ANALISIS EMOSI TWEET BERBAHASA INDONESIA MENGGUNAKAN NATURAL LANGUAGE PROCESSING (NLP) DAN DASHBOARD STREAMLIT

Disusun untuk Memenuhi Project Capstone
Program Coding Camp Data Science

DBS Foundation × Dicoding Indonesia

KELOMPOK:
CC26-PSU212

DISUSUN OLEH:

- SHAFIRA NABILA NOER POERWANTO (CDCC284D6X2715)
- NOVIA DJOEND LESTARI (CDCC284D6X2740)

A. IDE PROJECT

a. Judul Proyek Seribu Cerita

b. Latar Belakang

Perkembangan media sosial telah menghasilkan data teks dalam jumlah besar yang berisi opini, pengalaman, dan ekspresi emosi pengguna. Melalui tweet atau unggahan singkat, pengguna sering mengekspresikan berbagai emosi seperti *anger*, *fear*, *sad*, *happy* dan *neutral*. Informasi emosional tersebut dapat dimanfaatkan untuk memahami perilaku pengguna serta memperoleh gambaran mengenai kondisi emosional masyarakat secara lebih luas.

Dalam beberapa tahun terakhir, kesehatan mental menjadi isu yang semakin mendapat perhatian. Penelitian menunjukkan bahwa emosi yang diekspresikan melalui media sosial memiliki hubungan dengan kondisi psikologis seseorang, seperti stres, kecemasan, dan depresi (Zhang et al., 2023). Oleh karena itu, analisis emosi pada data media sosial memiliki potensi untuk mendukung pemahaman kondisi emosional pengguna secara lebih cepat dan objektif.

Namun, banyaknya data teks yang dihasilkan setiap hari membuat proses analisis secara manual menjadi tidak efisien. *Natural Language Processing* (NLP) hadir sebagai solusi untuk mengolah data teks secara otomatis dan mengekstraksi informasi yang terkandung di dalamnya. Salah satu penerapan NLP adalah klasifikasi emosi (*emotion classification*), yaitu proses mengidentifikasi kategori emosi yang terdapat pada suatu teks.

Berdasarkan hal tersebut, proyek ini dilakukan untuk mengeksplorasi dan menganalisis dataset tweet berbahasa Indonesia yang terdiri dari lima kategori emosi, yaitu *anger*, *fear*, *sad*, *happy* dan *neutral*. Analisis dilakukan untuk memahami distribusi emosi, karakteristik teks, serta keseimbangan data yang nantinya dapat menjadi dasar dalam pengembangan model klasifikasi emosi berbasis *Machine Learning*.

c. Tujuan

Proyek ini bertujuan untuk mengeksplorasi dan menganalisis dataset emosi pada tweet berbahasa Indonesia guna memahami karakteristik data, distribusi emosi, serta pola yang terdapat pada masing-masing kategori emosi. Adapun tujuan yang ingin dicapai adalah sebagai berikut:

1. Menganalisis distribusi dan proporsi data pada setiap kategori emosi (*anger*, *fear*, *sad*, *happy* dan *neutral*).
2. Mengidentifikasi proporsi emosi dominan dalam dataset
3. Menganalisis keseimbangan data antar label emosi yang dapat mempengaruhi proses pelatihan model
4. Mengeksplorasi karakteristik teks melalui *feature engineering* seperti jumlah kata dan panjang karakter tweet
5. Membandingkan pola karakteristik tweet pada emosi *happy* dan *sad*

d. Manfaat

Melalui proyek ini, diharapkan dapat diperoleh pemahaman yang lebih baik mengenai pola emosi pada tweet berbahasa Indonesia serta karakteristik data yang digunakan dalam analisis emosi. Adapun manfaat yang diharapkan adalah sebagai berikut:

1. Memberikan gambaran mengenai distribusi dan kecenderungan emosi pengguna media sosial berdasarkan kategori *anger*, *fear*, *sad*, *happy* dan *neutral*.
2. Membantu memahami karakteristik teks pada setiap kategori emosi melalui analisis fitur seperti jumlah kata dan panjang karakter tweet.
3. Menyediakan insight yang dapat digunakan sebagai dasar dalam pengembangan model klasifikasi emosi berbasis *Natural Language Processing* (NLP) dan *Machine Learning*.

B. TAHAPAN PROSES ANALISIS DATA

a. Pertanyaan Bisnis

1. Label emosi apa yang paling dominan pada tweet pengguna media sosial berdasarkan kategori *happy*, *fear*, *sad*, *anger* dan *neutral* dalam dataset, sehingga dapat menentukan fokus utama evaluasi emosi pengguna?
2. Bagaimana distribusi persentase tweet pada masing-masing label emosi *happy*, *fear*, *sad*, *anger* dan *neutral* untuk mengetahui kecenderungan emosi pengguna media sosial dalam dataset?
3. Apakah terdapat ketidakseimbangan jumlah data pada label emosi *happy*, *fear*, *sad*, *anger* dan *neutral* yang dapat mempengaruhi proses pelatihan model klasifikasi emosi?

b. Import Library

```
import pandas as pd
import numpy as np
from PIL import Image
from wordcloud import WordCloud, STOPWORDS, ImageColorGenerator
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
import re
import string
import nltk
import json
from statsmodels.stats.proportion import proportions_ztest
from scipy import stats
from scipy.stats import ttest_ind
```

Melakukan import library yang digunakan meliputi Pandas dan NumPy untuk pengolahan data, PIL, WordCloud, Matplotlib, dan Seaborn untuk visualisasi, Counter, re, string, dan NLTK untuk pemrosesan teks, serta JSON untuk membaca data berformat JSON. Selain itu, Statsmodels dan SciPy digunakan untuk melakukan analisis serta pengujian statistik.

c. Data Wrangling

i. Gathering Data

```
df_anger = pd.read_csv('AngerData.csv', sep='\t')
df_fear = pd.read_csv('FearData.csv', sep='\t')
df_sadness = pd.read_csv('SadData.csv', sep='\t')
df_joy = pd.read_csv('JoyData.csv', sep='\t')
df_neutral = pd.read_csv('NeutralData.csv', sep='\t')

data_1 = pd.concat([df_anger, df_fear, df_sadness, df_joy, df_neutral],
                    ignore_index=True)
data_1.to_csv('data_awal.csv', index=False)

data_1.head()
```

Pada tahap *Gathering Data*, dataset pertama diperoleh dari lima file terpisah, yaitu *AngerData.csv*, *FearData.csv*, *SadData.csv*, *JoyData.csv*, dan *NeutralData.csv*. Seluruh file dibaca menggunakan Pandas dan digabungkan menggunakan fungsi `pd.concat()` sehingga menghasilkan satu dataset yang disimpan sebagai `data_1`.



```
df_1 = pd.read_csv('raw_1.csv', sep='\t')
df_2 = pd.read_csv('raw_2.csv', sep='\t')
df_3 = pd.read_csv('raw_3.csv', sep='\t')

data_2 = pd.concat([df_1, df_2, df_3], ignore_index=True)
data_2.to_csv('data_awal2.csv', index=False)

data_2.head()
```

Selain itu, digunakan dataset kedua yang berasal dari tiga file mentah, yaitu *raw_1.csv*, *raw_2.csv*, dan *raw_3.csv*. Ketiga file tersebut juga digabungkan menggunakan `pd.concat()` dan disimpan sebagai *data_2*. Kedua dataset ini kemudian digunakan pada tahap *Assessing Data*, *EDA*, dan *Data Preprocessing*.

ii. Assessing Data



```
data_1.info()
data_1.isna().sum()
duplicate_count = data_1.duplicated().sum()
print("Jumlah data duplikat:", duplicate_count)
data_1['Label'].value_counts()
```

Pada tahap *Assessing Data*, dilakukan pemeriksaan terhadap struktur dan kualitas *data_1*, meliputi pengecekan tipe data, nilai kosong (*missing values*), data duplikat, serta distribusi label emosi. Tahap ini bertujuan untuk mengidentifikasi potensi permasalahan data sebelum dilakukan analisis dan preprocessing.



```
data_2.info()
data_2.isna().sum()
duplicate_count = data_2.duplicated().sum()
print("Jumlah data duplikat:", duplicate_count)
data_2['Label'].value_counts()
```

Selanjutnya, *data_2* diperiksa untuk mengetahui struktur data, nilai kosong, data duplikat, dan distribusi label emosi. Tahap ini dilakukan untuk memastikan kualitas data sebelum memasuki proses analisis dan preprocessing.

iii. Restructuring Data

```
data_1 = data_1.rename(columns={'Tweet': 'tweet'})
data_1 = data_1.rename(columns={'Label': 'label'})

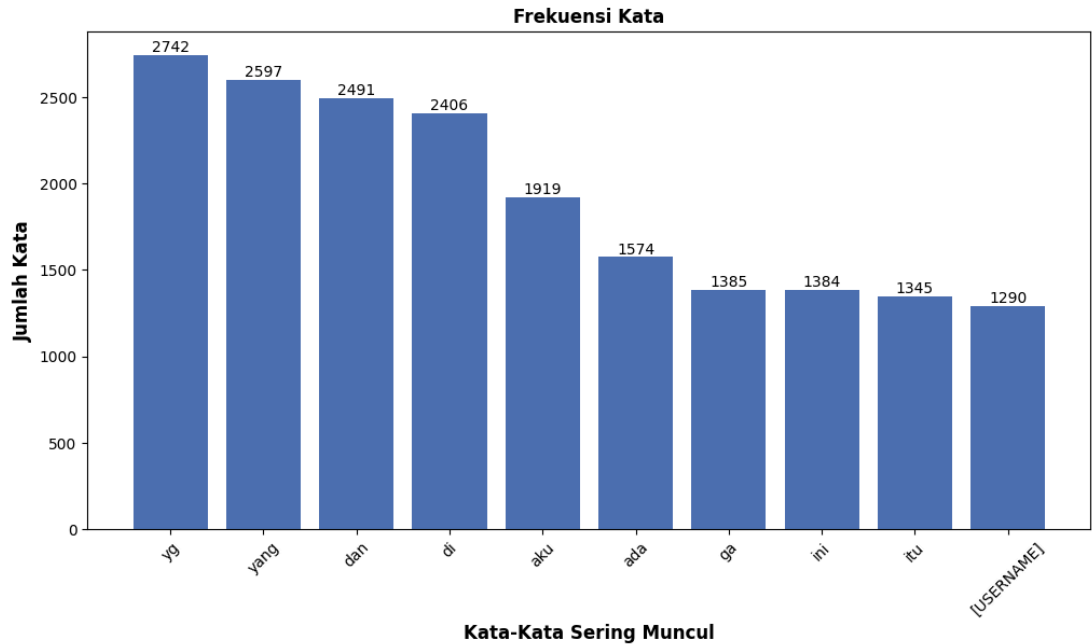
data_2[['label', 'tweet']] = data_2['label,tweet'].str.split(r'[:,]', n=1,
expand=True)
data_2 = data_2[['tweet', 'label']]

df_all = pd.concat([data_1, data_2], ignore_index=True)
print(df_all.shape)
print(df_all['label'].unique())
df_all.head()
```

Pada tahap *Data Restructuring*, dilakukan penyesuaian struktur kolom pada `data_1` dan `data_2` agar memiliki format yang seragam. Selanjutnya, kedua dataset digabungkan menjadi satu dataset utama yang siap digunakan untuk proses analisis dan preprocessing.

d. EDA sebelum pre processing data





Berdasarkan *wordcloud* dan frekuensi kata, dataset masih didominasi oleh *stopwords*, kata informal, serta *noise* seperti token *USERNAME*. Oleh karena itu, diperlukan tahap *preprocessing* untuk membersihkan dan menormalisasi teks sebelum digunakan dalam analisis maupun pemodelan NLP.

e. Pre processing Data

i. Lower case

```
df_all['tweet_asli'] = df_all['tweet']
df_all['tweet'] = df_all['tweet'].str.lower()
df_all['label'] = df_all['label'].str.lower()

df_all.head()
```

Pada tahap *case folding*, seluruh teks pada kolom tweet dan label diubah menjadi huruf kecil (*lowercase*) untuk menyeragamkan penulisan data.

ii. Label normalization

```
df_all['label'] = df_all['label'].replace('joy', 'happy')
df_all['label'] = df_all['label'].replace('sadness', 'sad')
df_all = df_all.drop(df_all[df_all['label'] == 'love'].index)

print(df_all['label'].unique())
df_all.head()
```

Pada tahap penyesuaian label, kategori joy diubah menjadi *happy* dan *sadness* diubah menjadi *sad* untuk menyeragamkan label emosi. Selain itu, data dengan

label love dihapus sehingga dataset hanya terdiri dari lima kategori emosi, yaitu *anger*, *fear*, *sad*, *happy* dan *neutral*.

iii. Remove mention



```
df_all['tweet'] = df_all['tweet'].str.replace(r'@\w+', '', regex=True)
df_all.head()
```

Pada tahap *remove mention*, dilakukan penghapusan mention pengguna (misalnya @username) dari teks tweet menggunakan *regular expression*. Proses ini bertujuan untuk mengurangi *noise* dan mempertahankan informasi yang lebih relevan untuk analisis.

iv. Remove url



```
def remove_url(text):
    return re.sub(r'http\S+|www\.\S+', '', text)

df_all['tweet'] = df_all['tweet'].apply(remove_url)
df_all.head()
```

Pada tahap *remove URL*, seluruh tautan atau alamat website dihapus dari teks tweet untuk mengurangi *noise* dan mempertahankan informasi yang relevan.

v. Remove hashtag



```
def remove_hashtags(text):
    return re.sub(r'#\w+', '', text)

df_all['tweet'] = df_all['tweet'].apply(remove_hashtags)
df_all.head()
```

Pada tahap *remove hashtag*, seluruh hashtag (#kata) dihapus dari teks tweet untuk mengurangi *noise* pada data.

vi. Word repetition normalization



```
def handle_repetition(text):
    return re.sub(r'([a-zA-Z]+)2', r'\1 \1', text)

df_all['tweet'] = df_all['tweet'].apply(handle_repetition)
df_all.head()
```

Pada tahap *word repetition normalization*, kata yang ditulis berulang menggunakan angka 2 (misalnya *anak2*) diubah menjadi bentuk yang lebih jelas (*anak anak*) agar lebih mudah diproses pada tahap analisis teks.

vii. Remove punctuation

```
def remove_punctuation(text):  
    return text.translate(str.maketrans('', '', string.punctuation))  
  
df_all['tweet'] = df_all['tweet'].apply(remove_punctuation)  
df_all.head()
```

Pada tahap *remove punctuation*, seluruh tanda baca dihapus dari teks tweet untuk menghasilkan teks yang lebih bersih dan seragam.

viii. Remove symbols

```
def remove_symbols(text):  
    text = re.sub(r'^a-zA-Z0-9\s', '', text)  
    return text  
  
df_all['tweet'] = df_all['tweet'].apply(remove_symbols)  
df_all.head()
```

Pada tahap *remove symbols*, karakter atau simbol non-alfanumerik yang tidak diperlukan dihapus dari teks tweet untuk mengurangi *noise* pada data.

ix. Character normalize

```
def normalize_repeated_chars(text):  
    return re.sub(r'(\.)\1{2,}', r'\1', text)  
  
df_all['tweet'] = df_all['tweet'].apply(normalize_repeated_chars)  
df_all.head()
```

Pada tahap *character normalization*, karakter yang berulang secara berlebihan pada kata dikurangi menjadi satu karakter untuk menyeragamkan penulisan teks.

x. Normalization of non standard words

```
nltk.download('punkt')  
dictionary_df = pd.read_csv("kata_baku.csv")  
  
spelling_correction = dict(zip(dictionary_df['tidak_baku'],  
                                dictionary_df['kata_baku']))  
def correct_spelling(text, corrections):  
    words = text.split()  
    corrected_words = [corrections.get(word, word) for word in words]  
    return ' '.join(corrected_words)  
  
df_all['tweet'] = df_all['tweet'].apply(lambda x: correct_spelling(x,  
                                                                    spelling_correction))  
df_all.head()
```

Pada tahap *normalization of non-standard words*, kata tidak baku diubah menjadi bentuk baku menggunakan kamus normalisasi untuk menyeragamkan penulisan kata pada teks tweet.

xi. Slang normalization

```
slang_words_file = 'combined_slang_words.txt'

try:
    with open(slang_words_file, 'r', encoding='utf-8') as file:
        slang_words_dict = json.load(file)

except FileNotFoundError:
    slang_words_dict = {}
    print(f"Warning: File '{slang_words_file}' tidak ditemukan (JSON slang kosong).")

try:
    slang_csv = pd.read_csv('slang_indo_fix.csv')
    slang_csv_dict = dict(zip(slang_csv.iloc[:, 0], slang_csv.iloc[:, 1]))

except FileNotFoundError:
    slang_csv_dict = {}
    print("Warning: slang_indo_fix.csv tidak ditemukan.")

slang_words_dict.update(slang_csv_dict)

def normalize_slang(text, slang_dict):
    if isinstance(text, str):
        words = text.split()
        normalized_words = [slang_dict.get(word, word) for word in words]
        return ' '.join(normalized_words)
    return text

df_all['tweet'] = df_all['tweet'].apply(
    lambda x: normalize_slang(x, slang_words_dict)
)

df_all.head()
```

Pada tahap *slang normalization*, kata-kata slang atau bahasa tidak formal diubah menjadi bentuk yang lebih baku menggunakan kamus slang untuk menyeragamkan penulisan teks.

xii. Remove duplicates

```
df_all.drop_duplicates(subset = 'tweet', keep = 'first', inplace = True)
df_all.head()
```

Pada tahap *remove duplicates*, tweet duplikat dihapus menggunakan `drop_duplicates()` sehingga hanya data unik yang dipertahankan

xiii. Remove invalid words

```
def remove_no_vowel_words(text):
    words = text.split()
    clean_words = [word for word in words if re.search(r'[aiueo]', word)]
    return ' '.join(clean_words)

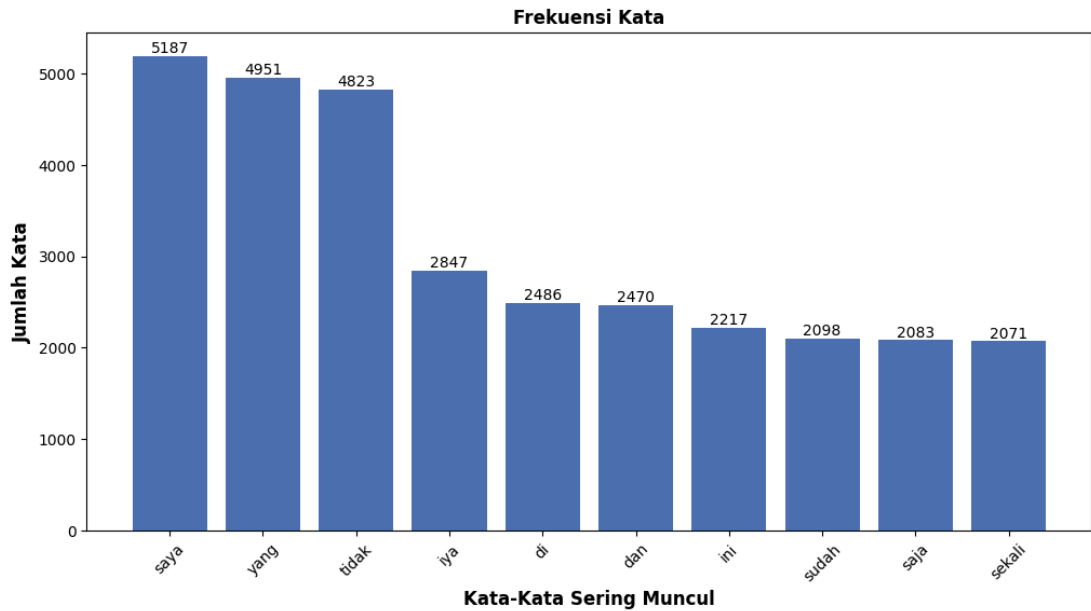
df_all['tweet'] = df_all['tweet'].apply(remove_no_vowel_words)
df_all.head()
```

Pada tahap *remove no vowel words*, kata yang tidak memiliki huruf vokal (a, i, u, e, o) dihapus dari tweet menggunakan regex, sehingga hanya kata yang bermakna dan valid yang dipertahankan.

f. EDA setelah pre processing



Tweet digabung menjadi satu teks dan nilai kosong diisi dengan fillna(""). Setelah itu teks diproses untuk membentuk WordCloud yang menampilkan kata-kata yang paling sering muncul dalam dataset.

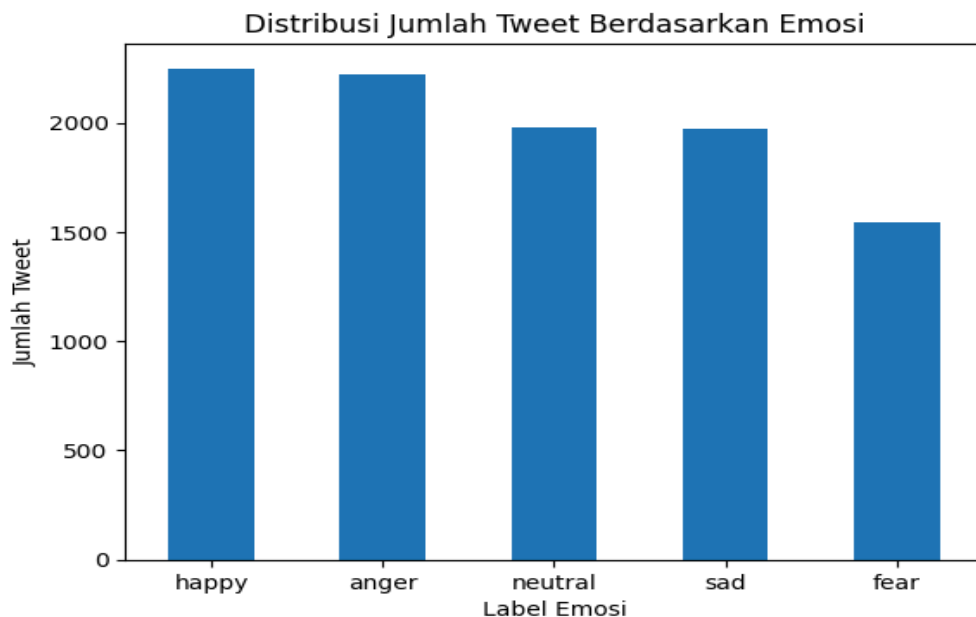


Selanjutnya, teks dipecah menjadi token menggunakan `split()`, kemudian dihitung frekuensi kemunculan setiap kata menggunakan Counter. Dari hasil tersebut diambil 10 kata teratas (top words), lalu divisualisasikan dalam bentuk diagram batang untuk melihat kata yang paling dominan secara lebih ringkas.

g. Visualisasi Explanatory

i. Pertanyaan 1

Label emosi apa yang paling dominan pada tweet pengguna media sosial berdasarkan kategori *happy*, *fear*, *sad*, *anger* dan *neutral* dalam dataset, sehingga dapat menentukan fokus utama evaluasi emosi pengguna?

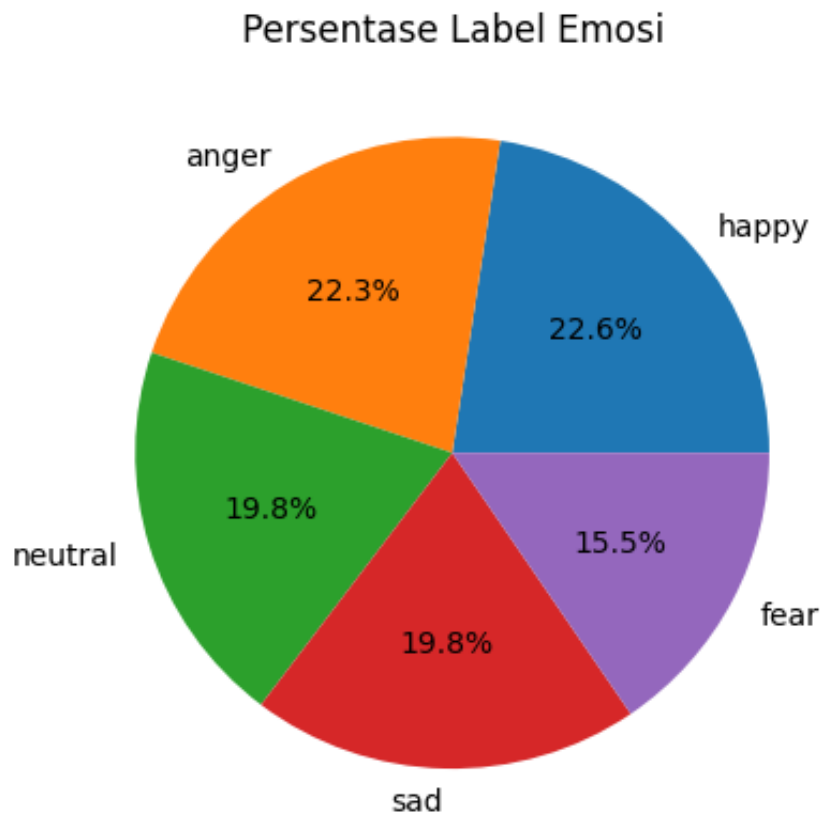


Berdasarkan hasil perhitungan menggunakan `value_counts()` pada kolom label, diperoleh jumlah masing-masing emosi dalam dataset. Nilai tersebut kemudian digunakan untuk menentukan emosi yang paling dominan dengan fungsi `idxmax()`, yaitu emosi dengan jumlah tweet terbanyak. Hasil ini divisualisasikan dalam bentuk diagram batang agar perbandingan antar kelas emosi lebih mudah dipahami.

Dari grafik tersebut terlihat bahwa emosi *happy* merupakan label yang paling dominan, diikuti oleh *anger* yang jumlahnya juga cukup tinggi. Sementara itu, emosi *neutral* dan *sad* memiliki jumlah yang relatif seimbang, sedangkan *fear* menjadi kategori dengan jumlah tweet paling sedikit. Hal ini menunjukkan bahwa distribusi data tidak sepenuhnya merata, sehingga terdapat indikasi ketidakseimbangan kelas yang dapat berpengaruh pada proses analisis atau pemodelan selanjutnya.

ii. Pertanyaan 2

Bagaimana distribusi persentase tweet pada masing-masing label emosi *happy*, *fear*, *sad*, *anger*, dan *neutral* untuk mengetahui kecenderungan emosi pengguna media sosial dalam dataset?

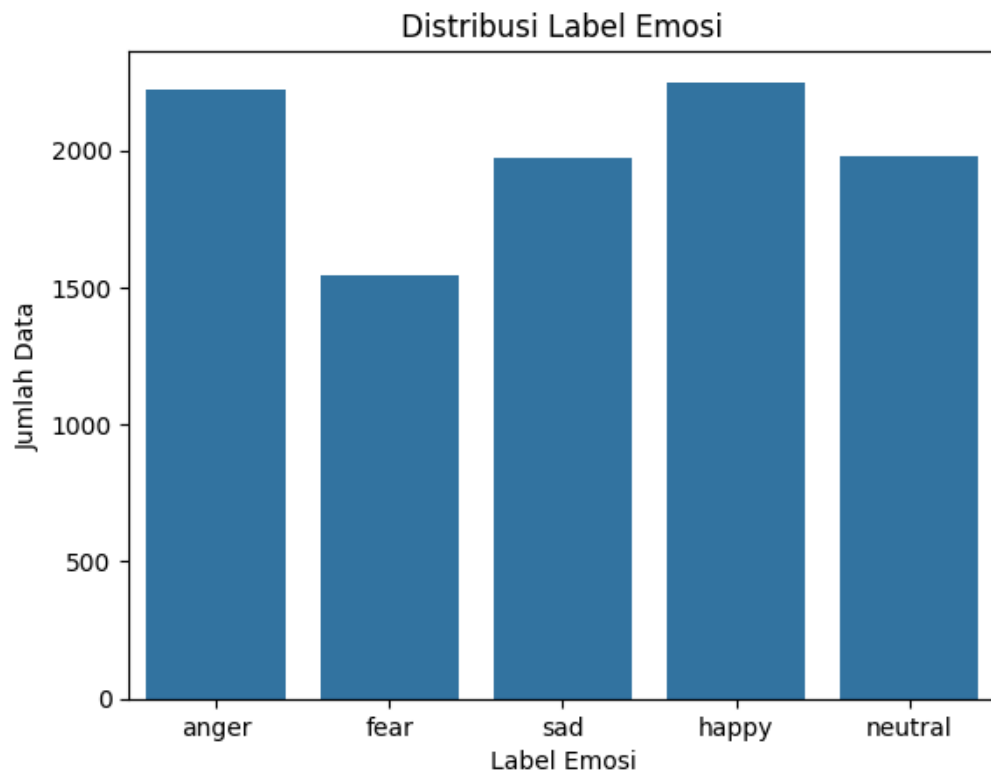


Berdasarkan hasil perhitungan persentase label emosi menggunakan `value_counts(normalize=True) * 100`, diperoleh proporsi masing-masing kategori emosi dalam bentuk persentase. Hasil perhitungan tersebut kemudian disimpan dalam DataFrame dan divisualisasikan menggunakan diagram pie (`plot(kind='pie')`) untuk mempermudah pengamatan terhadap distribusi emosi dalam dataset.

Emosi *happy* memiliki persentase tertinggi sebesar 22,6%, diikuti *anger* sebesar 22,3%. Sementara itu, emosi *sad* dan *neutral* masing-masing sebesar 19,8%, sedangkan *fear* memiliki persentase terendah yaitu 15,5%. Hasil ini menunjukkan bahwa tweet dalam dataset cenderung didominasi oleh emosi *happy* dan *anger*, namun secara keseluruhan distribusi antar kategori emosi cukup seimbang sehingga mampu merepresentasikan berbagai ekspresi emosi pengguna media sosial dengan baik.

iii. Pertanyaan 3

Apakah terdapat ketidakseimbangan jumlah data pada label emosi *happy*, *fear*, *sad*, *anger*, dan *neutral* yang dapat mempengaruhi proses pelatihan model klasifikasi emosi?



Berdasarkan hasil perhitungan jumlah data pada setiap label emosi menggunakan `value_counts()` dan visualisasi `countplot()`, dilakukan analisis untuk mengetahui apakah terdapat ketidakseimbangan data antar kategori

emosi. Selain menampilkan jumlah data pada masing-masing label, kode juga menghitung rasio antara jumlah data terbesar dan terkecil sebagai indikator tingkat keseimbangan dataset.

Grafik menunjukkan bahwa label *happy* memiliki jumlah data terbanyak, sedangkan *fear* memiliki jumlah data paling sedikit. Namun, perbedaan jumlah data antar label tidak terlalu jauh sehingga distribusinya masih relatif merata. Dengan rasio terbesar dan terkecil yang berada di bawah batas 1,5, dataset dapat disimpulkan cukup seimbang dan tidak berpotensi memberikan bias yang signifikan pada proses pelatihan model klasifikasi emosi.

h. Feature Engineering

i. Label encoding

```
● ● ●  
  
label_mapping = {  
    'anger': 0,  
    'fear': 1,  
    'sad': 2,  
    'neutral': 3,  
    'happy': 4  
}  
  
data['label_id'] = data['label'].map(label_mapping)  
data.head()
```

Pada tahap *feature engineering*, dilakukan *label encoding* dengan mengubah setiap kategori emosi menjadi representasi numerik, yaitu *anger* = 0, *fear* = 1, *sad* = 2, *neutral* = 3, dan *happy* = 4. Hasil transformasi disimpan pada kolom *label_id* agar dapat digunakan dalam proses pemodelan Machine Learning.

ii. Word Count

```
● ● ●  
  
data['jumlah_kata'] = data['tweet'].apply(  
    lambda x: len(str(x).split()))  
  
data.head()
```

Pada tahap *feature engineering*, ditambahkan fitur *jumlah_kata* yang merepresentasikan jumlah kata pada setiap tweet. Fitur ini digunakan untuk memberikan informasi tambahan mengenai karakteristik panjang teks.

iii. Character Count

```
data['panjang_karakter'] = data['tweet'].apply(
    lambda x: len(str(x)))

data.head()
```

Pada tahap *feature engineering*, ditambahkan fitur `panjang_karakter` yang merepresentasikan jumlah karakter pada setiap tweet. Fitur ini digunakan untuk menggambarkan panjang teks dan dapat dimanfaatkan dalam proses analisis maupun pemodelan.

i. A/B Testing

```
from scipy.stats import ttest_ind

group_A = data[data['label'] == 'happy']['jumlah_kata']
group_B = data[data['label'] == 'sad']['jumlah_kata']

print("Rata-rata jumlah kata Happy :", round(group_A.mean(), 2))
print("Rata-rata jumlah kata Sad   :", round(group_B.mean(), 2))

t_stat, p_value = ttest_ind(group_A, group_B)

print("\nHASIL T-TEST ")
print(f"T-Statistic : {t_stat:.4e}")
print(f"P-Value : {p_value:.4e}")

alpha = 0.05

print("\nINTERPRETASI")

if p_value < alpha:
    print("H0 ditolak. Terdapat perbedaan signifikan antara emosi happy dan sad, sehingga menunjukkan bahwa jumlah kata pada kedua emosi memiliki perbedaan yang signifikan.")
else:
    print("H0 diterima. Tidak terdapat perbedaan signifikan antara emosi happy dan sad.")
```

A/B Testing dilakukan menggunakan Independent Sample T-Test untuk membandingkan rata-rata jumlah kata pada tweet berlabel *happy* dan *sad*. Berdasarkan hasil perhitungan, tweet dengan emosi *happy* memiliki rata-rata 19,14 kata, sedangkan tweet dengan emosi *sad* memiliki rata-rata 22,38 kata. Hasil ini menunjukkan bahwa tweet yang mengekspresikan emosi sedih cenderung ditulis lebih panjang dibandingkan tweet yang mengekspresikan emosi bahagia.

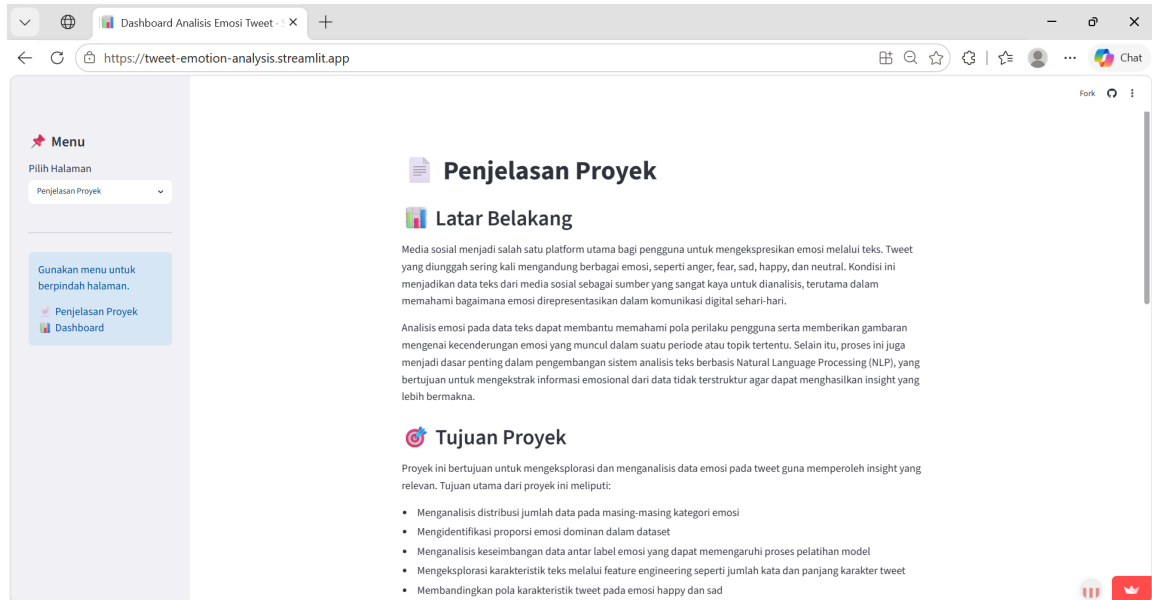
Hasil uji statistik menghasilkan T-Statistic sebesar -7,7997 dan P-Value sebesar $7,7656 \times 10^{-15}$. Karena nilai P-Value lebih kecil dari 0,05, maka H_0 ditolak, yang berarti terdapat perbedaan signifikan antara jumlah kata pada tweet berlabel *happy*

dan *sad*. Temuan ini menunjukkan bahwa fitur jumlah_kata memiliki potensi untuk digunakan sebagai salah satu fitur pendukung dalam analisis maupun klasifikasi emosi.

C. DASHBOARD STREAMLIT

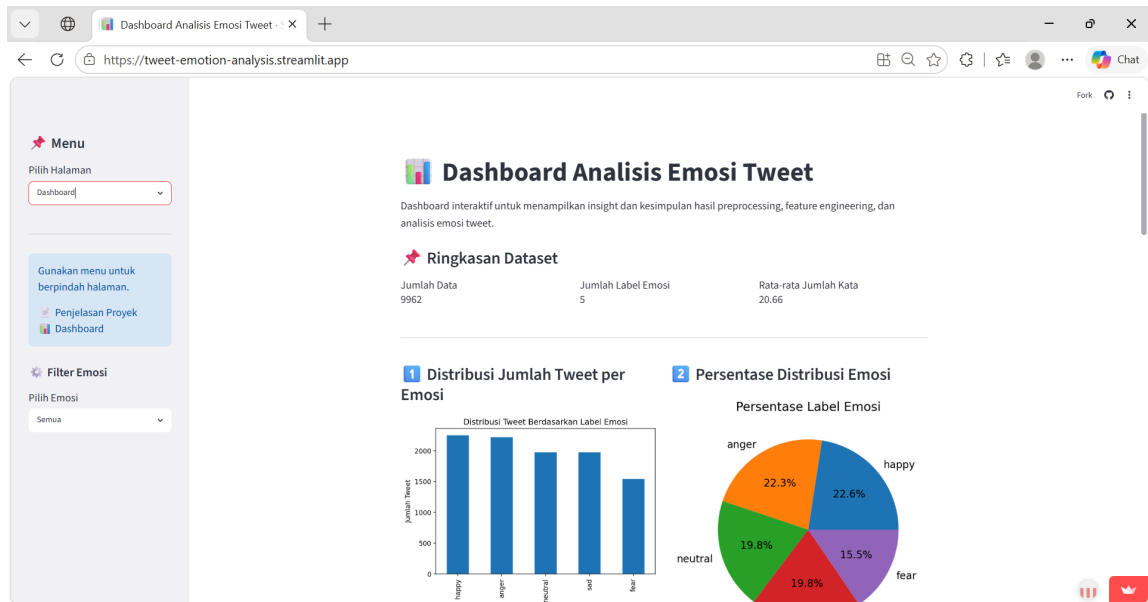
Link Dashboard: <https://tweet-emotion-analysis.streamlit.app/>

Tampilan Halaman Penjelasan Proyek



Halaman Penjelasan Proyek berfungsi sebagai pengantar yang memberikan informasi mengenai latar belakang dan tujuan analisis emosi pada data tweet. Pada halaman ini dijelaskan alasan pemilihan topik, pentingnya analisis emosi dalam memahami ekspresi pengguna media sosial, serta tujuan yang ingin dicapai, seperti mengidentifikasi emosi dominan, menganalisis distribusi setiap label emosi, mengevaluasi keseimbangan data, dan mengeksplorasi karakteristik tweet. Selain itu, tersedia sidebar navigasi yang memudahkan pengguna berpindah ke halaman dashboard untuk melihat hasil analisis secara lebih rinci.

Tampilan Halaman Dashboard Analisis Emosi Tweet



Halaman Dashboard Analisis Emosi Tweet menampilkan ringkasan dan visualisasi hasil analisis data secara interaktif. Pada bagian atas terdapat Ringkasan Dataset yang menyajikan informasi penting seperti jumlah data, jumlah label emosi, dan rata-rata jumlah kata per tweet. Selain itu, dashboard menampilkan visualisasi distribusi jumlah tweet per emosi dalam bentuk diagram batang dan persentase distribusi emosi dalam bentuk diagram pie untuk memudahkan pengguna memahami sebaran data pada setiap kategori emosi. Di sisi kiri juga tersedia fitur filter emosi yang memungkinkan pengguna memilih kategori emosi tertentu sehingga analisis dapat dilakukan secara lebih spesifik dan interaktif.

D. KESIMPULAN

Proyek Seribu Cerita berhasil menganalisis emosi pada tweet berbahasa Indonesia melalui tahapan pengumpulan data, preprocessing, eksplorasi data, visualisasi, dan feature engineering. Hasil analisis menunjukkan bahwa emosi *happy* merupakan kategori yang paling dominan dalam dataset, sedangkan emosi *fear* memiliki proporsi paling rendah. Distribusi data antar kategori emosi juga tergolong cukup seimbang sehingga layak digunakan untuk analisis dan pengembangan model klasifikasi emosi. Selain itu, hasil *A/B Testing* menunjukkan bahwa tweet dengan emosi *sad* cenderung memiliki jumlah kata lebih banyak dibandingkan tweet dengan emosi *happy*. Secara keseluruhan, proyek ini memberikan gambaran mengenai pola emosi dan karakteristik tweet yang dapat menjadi dasar pengembangan sistem klasifikasi emosi berbasis NLP.

E. DAFTAR PUSTAKA

1. Aggarwal, C. C. (2018). *Machine Learning for Text*. Cham: Springer International Publishing.
2. Jurafsky, D., & Martin, J. H. (2024). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition* (3rd ed.). Pearson.
3. Zhang, Y., Lyu, H., Liu, Y., Zhang, X., Wang, Y., & Luo, J. (2023). *Social Media-Based Mental Health Detection: A Survey*. *Information Fusion*, 91, 312–334.

F. LAMPIRAN

Drive Project :  Data Science Capstone
Code :  Capstone Data Science.ipynb
ZIP Streamlit : [streamlit.zip](#)